
Integration Manual

for MPC560XB GPT Driver

Document Number: IM13GPTASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	8
2.2	Acronyms and Definitions.....	8
2.3	Reference List.....	9
Chapter 3		
Building the Driver		
3.1	Build Options.....	11
3.1.1	GHS Compiler/Linker/Assembler Options.....	11
3.1.2	DIAB Compiler/Linker/Assembler Options.....	13
3.1.3	CW Compiler/Linker/Assembler Options.....	14
3.1.4	HT Compiler/Linker/Assembler Options.....	16
3.2	Files required for Compilation.....	17
3.3	Setting up the Plug-ins.....	20
Chapter 4		
Function Calls to Module		
4.1	Function Calls during Start-up.....	21
4.2	Function Calls during Shutdown.....	21
4.3	Function Calls during Wake-up.....	21
Chapter 5		
Module Requirements		
5.1	Peripheral Hardware Requirements.....	23
5.2	ISR to configure within OS – dependencies.....	23
5.3	ISR macro.....	25
5.4	Other AUTOSAR modules - Dependencies.....	26
5.4.1	Development Error Tracer:	26

Section number	Title	Page
5.4.2	Diagnostic Event Manager:.....	26
5.4.3	ECU Manager:.....	26
5.4.4	MCU module:.....	26
5.4.5	Configuration dependency to other module:	26
5.5	Exclusive areas to be defined in BSW scheduler.....	27
5.5.1	EVEN_EXCLUSIVE_AREA	27
5.5.2	ODD_EXCLUSIVE_AREA.....	27
5.5.3	GPT_EXCLUSIVE_AREA_64.....	27

Chapter 6 Main API Requirements

6.1	Main functions calls within BSW scheduler.....	29
6.2	Calls to notification functions, callbacks, callouts.....	29
6.2.1	Call-back Notifications:.....	29
6.2.2	User Notification:.....	29

Chapter 7 Memory Allocation

7.1	Sections to be defined in MemMap.h.....	31
7.2	Linker command file.....	33

Chapter 8 CONFIGURATION PARAMETER CONSIDERATIONS

Chapter 9 Integration Steps

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	17.02.2012	Surjit Kumar	Update for Beta 0.9.0
1.1	14.05.2013	Subramanya MN	Update for RTM 1.0.0
1.2	06.06.2014	Nguyen Viet Hung	Update for MPC560XB RTM 1.0.1 Release



Chapter 2

Introduction

This Integration Manual describes the integration requirements for Autosar GPT Driver for

Freescale Semiconductor's MPC560XB microcontrollers .

The roadmap for the document is as follows:

Building the Driver

This section gives a brief overview of the build procedure (compiler, linker options and source files) and Plugins setup.

Function Calls to Module

This section lists the various function calls to modules during Start-up, Shutdown and Wake-up.

Module Requirements

This section specifies the various module requirements related to

- Exclusive areas to be defined in BSW scheduler
- Peripheral Hardware Requirements
- Specific interface to other modules
- ISR to configure within OS
- Dependencies with other AUTOSAR modules

Main API Requirements

This section specifies the requirements related to the main GPT_main API and gives a brief overview of the main functions calls within BSW scheduler, API_Name Requirements and calls to notification functions, callbacks, callouts.

Memory Allocation

This section describes the memory allocation requirements namely the sections to be defined in MemMap.h and the linker command file.

CONFIGURATION PARAMETER CONSIDERATIONS

This section covers the various Pre Compile, Link Time and Post Build time configuration parameters.

Integration Steps

This section describes in brief the steps for integrating GPT module.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor :

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkl_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	---

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
GPT	General Purpose Timer
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit
STM	System Timer Module
PIT	Periodic Interrupt Timer
RTI	Real Time Interrupt
API	Autonomous Periodic Interrupt

2.3 Reference List

Table 2-3. Reference List

Title	Version
AUTOSAR GPT Driver Software Specification Document	R3.2.0 Rev0003
MPC5607B Reference Manual	Rev. 7.2, May 2012
MPC5604B Reference Manual	Rev. 8.2, September 2013
5606B Reference Manual	Rev. 2, May 2014
5602D Reference Manual	Rev. 4.2, Dec 2013
Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3

Building the Driver

This section describes the source files and various compilers, linker options used for building the Gpt driver. It also explains the EB tresos Studio plugin setup procedure.

3.1 Build Options

The driver files are compiled using

Green Hills MULTI v6.1.4

Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511

Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224

HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

3.1.1 GHS Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.

Table continues on the next page...

Table 3-1. Compiler Options (continued)

Option	Description
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-3. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 CW Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler

Table continues on the next page...

Table 3-7. Compiler Options (continued)

Option	Description
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-9. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size

Table continues on the next page...

Table 3-9. Linker Options (continued)

Option	Description
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,'. Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script file sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the Autosar GPT driver for Freescale Semiconductor's MPC560XB microcontroller.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same Autosar major and minor versions can be compiled.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same Autosar major and minor versions can be compiled.

GPT Files:

```
..\Gpt_TS_T2D13M10I1R0\src\Stm_Gpt_LLD_IRQ.c
..\Gpt_TS_T2D13M10I1R0\src\Stm_Gpt_LLD.c
..\Gpt_TS_T2D13M10I1R0\src\Reg_eSys_eMIOS_Gpt.c
..\Gpt_TS_T2D13M10I1R0\src\Pit_Gpt_LLD_IRQ.c
..\Gpt_TS_T2D13M10I1R0\src\Pit_Gpt_LLD.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_NonASR.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_LLD.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_Irq.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_eMIOS_Irq.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_eMIOS_LLD.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_Rtc_LLD.c
..\Gpt_TS_T2D13M10I1R0\src\Gpt_Rtc_LLD_Irq.c

..\Gpt_TS_T2D13M10I1R0\include\Stm_Gpt_LLD_CfgEx.h
..\Gpt_TS_T2D13M10I1R0\include\Stm_Gpt_LLD.h
..\Gpt_TS_T2D13M10I1R0\include\Reg_eSys_STM.h
..\Gpt_TS_T2D13M10I1R0\include\Reg_eSys_PIT.h
..\Gpt_TS_T2D13M10I1R0\include\Reg_eSys_eMIOS_CfgEx.h
..\Gpt_TS_T2D13M10I1R0\include\Reg_eSys_eMIOS.h
..\Gpt_TS_T2D13M10I1R0\include\Pit_Gpt_LLD_CfgEx.h
..\Gpt_TS_T2D13M10I1R0\include\Pit_Gpt_LLD.h
..\Gpt_TS_T2D13M10I1R0\include\Gpt_NonASR.h
..\Gpt_TS_T2D13M10I1R0\include\Gpt_LLD.h
..\Gpt_TS_T2D13M10I1R0\include\Gpt.h
```

```

..\Gpt_TS_T2D13M10I1R0\include\Gpt_eMIOS_LLD_CfgEx.h
..\Gpt_TS_T2D13M10I1R0\include\Gpt_eMIOS_LLD.h
..\Gpt_TS_T2D13M10I1R0\include\Gpt_Rtc_LLD.h
..\Gpt_TS_T2D13M10I1R0\include\Gpt_Rtc_LLD_CfgEx.h
..\Gpt_TS_T2D13M10I1R0\include\Reg_eSys_Rtc.h

```

Gpt_Cfg.c (For PC Variant) - This file should be generated by the user using a configuration tool for compilation

Gpt_PBCfg.c (For PB Variant) - This file should be generated by the user using a configuration tool for compilation

Gpt_Cfg.h - This file should be generated by the user using a configuration tool for compilation

Other include files:

Files from Base folder:

```

..\Base_TS_T2D13M10I1R0\include\MemMap.h
..\Base_TS_T2D13M10I1R0\include\Platform_Types.h
..\Base_TS_T2D13M10I1R0\include\Std_Types.h
..\Base_TS_T2D13M10I1R0\include\Register.h
..\Base_TS_T2D13M10I1R0\include\Reg_eSys.h
..\Base_TS_T2D13M10I1R0\include\Reg_Macros.h
..\Base_TS_T2D13M10I1R0\include\Cer.h

```

Files from Det folder:

```

..\Det_TS_T2D13M10I1R0\include\Det.h

```

Files from Rte folder:

```

..\Rte_TS_T2D13M10I1R0\include\Rte_Gpt.h

```

Files from EcuM folder:

```

..\EcuM_TS_T2D13M10I1R0\include\EcuM.h
..\EcuM_TS_T2D13M10I1R0\include\EcuM_Cbk.h

```

3.3 Setting up the Plug-ins

All the Autosar MCAL drivers for MPC560XB were designed to be configured using Tresos® Studio configuration and code generation tool from Elektrobit Tresos Release 14.2.1 (b140128-1223).

Location of various files inside the plugin folder is explained below.

Module Parameter Definition File:

..\Gpt_TS_T2D13M10I1R0\config\Gpt.xdm

..\EcuM_TS_T2D13M10I1R0\config\EcuM.xdm

Code Generation Templates for Pre-Compile time configuration parameters:

..\Gpt_TS_T2D13M10I1R0\generate_PC\src\Gpt_Cfg.c

..\Gpt_TS_T2D13M10I1R0\generate_PC\include\Gpt_Cfg.h

..\EcuM_TS_T2D13M10I1R0\generate_PC\include\EcuM_Cfg.h

Code Generation Templates for Post-Build time configuration parameters:

..\Gpt_TS_T2D13M10I1R0\generate_PB\src\Gpt_PBcfg.c

Tresos Configuration tool files:

..\Gpt_TS_T2D13M10I1R0\META-INF\MANIFEST.MF

..\Gpt_TS_T2D13M10I1R0\plugin.xml

Steps to generate configurations:

1. Copy the module folder (Gpt_TS_T2D13M10I1R0), (EcuM_TS_T2D13M10I1R0) into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the Tresos GUI to modify configuration parameters values.
4. Generate the Pre-Compile and Post-Build files.

Chapter 4

Function Calls to Module

4.1 Function Calls during Start-up

GPT shall be initialized during STARTUP1 phase of EcuM initialization. The API to be called for this is Gpt_Init() MCU module shall be initialized before GPT is initialized.

4.2 Function Calls during Shutdown

If GptWakeupFunctionalityApi and GptWakeupSourceRef are enabled, Gpt_SetMode(GPT_MODE_SLEEP) API shall be called during GO SLEEP phase of EcuM to configure the hardware for Sleep mode.

4.3 Function Calls during Wake-up

For the platforms where the GPT driver controls wakeup hw sources, if the GptWakeupFunctionalityApi and GptWakeupSourceRef are enabled, the driver shall report the wakeup event to EcuM through EcuM_SetWakeupEvent(Source) upon the hw source event.

Chapter 5

Module Requirements

5.1 Peripheral Hardware Requirements

The driver can implement maximum 77 channels on maximum 5 MPC560XB peripherals.

- maximum 64 channels are implemented on 2 Enhanced Modular IO Subsystem (eMIOS) modules depending on the derivative of the controller.
- 4 channels are implemented on System Timer Module (STM).
- 8 channels are implemented on Periodic Interrupt Timer (PIT).
- 1 Real Time Clock (RTC) channel.

Refer Table GPT Hardware Channel availability for MPC560XB family in User Manual

5.2 ISR to configure within OS – dependencies

The following ISR's are used by the GPT driver:

Table 5-1. GPT ISR's

ISR Name	Hardware interrupt vector
For EMIOS0	
ISR(EMIOS_0_CH_0_CH_1_ISR)	141
ISR(EMIOS_0_CH_2_CH_3_ISR)	142
ISR(EMIOS_0_CH_4_CH_5_ISR)	143
ISR(EMIOS_0_CH_6_CH_7_ISR)	144
ISR(EMIOS_0_CH_8_CH_9_ISR)	145
ISR(EMIOS_0_CH_10_CH_11_ISR)	146
ISR(EMIOS_0_CH_12_CH_13_ISR)	147
ISR(EMIOS_0_CH_14_CH_15_ISR)	148

Table continues on the next page...

Table 5-1. GPT ISR's (continued)

ISR Name	Hardware interrupt vector
ISR(EMIOS_0_CH_16_CH_17_ISR)	149
ISR(EMIOS_0_CH_18_CH_19_ISR)	150
ISR(EMIOS_0_CH_20_CH_21_ISR)	151
ISR(EMIOS_0_CH_22_CH_23_ISR)	152
ISR(EMIOS_0_CH_24_CH_25_ISR)	153
ISR(EMIOS_0_CH_26_CH_27_ISR)	154
ISR(EMIOS_0_CH_28_CH_29_ISR)	155
ISR(EMIOS_0_CH_30_CH_31_ISR)	156
For EMIOS1	
ISR(EMIOS_1_CH_0_CH_1_ISR)	157
ISR(EMIOS_1_CH_2_CH_3_ISR)	158
ISR(EMIOS_1_CH_4_CH_5_ISR)	159
ISR(EMIOS_1_CH_6_CH_7_ISR)	160
ISR(EMIOS_1_CH_8_CH_9_ISR)	161
ISR(EMIOS_1_CH_10_CH_11_ISR)	162
ISR(EMIOS_1_CH_12_CH_13_ISR)	163
ISR(EMIOS_1_CH_14_CH_15_ISR)	164
ISR(EMIOS_1_CH_16_CH_17_ISR)	165
ISR(EMIOS_1_CH_18_CH_19_ISR)	166
ISR(EMIOS_1_CH_20_CH_21_ISR)	167
ISR(EMIOS_1_CH_22_CH_23_ISR)	168
ISR(EMIOS_1_CH_24_CH_25_ISR)	169
ISR(EMIOS_1_CH_26_CH_27_ISR)	170
ISR(EMIOS_1_CH_28_CH_29_ISR)	171
ISR(EMIOS_1_CH_30_CH_31_ISR)	172
For PIT Timers	
ISR(Gpt_PIT_TIMER_0_ISR)	59
ISR(Gpt_PIT_TIMER_1_ISR)	60
ISR(Gpt_PIT_TIMER_2_ISR)	61
ISR(Gpt_PIT_TIMER_3_ISR)	127
ISR(Gpt_PIT_TIMER_4_ISR)	128
ISR(Gpt_PIT_TIMER_5_ISR)	129
ISR(Gpt_PIT_TIMER_6_ISR)	130
ISR(Gpt_PIT_TIMER_7_ISR)	131
For STM Timers	
ISR(Gpt_STM_Ch_0_ISR)	30
ISR(Gpt_STM_Ch_1_ISR)	31
ISR(Gpt_STM_Ch_2_ISR)	32
ISR(Gpt_STM_Ch_3_ISR)	33

Table continues on the next page...

Table 5-1. GPT ISR's (continued)

ISR Name	Hardware interrupt vector
For RTC Timer	
ISR(Gpt_RTC_Ch_0_ISR)	39

NOTE: EMIOS1 is not available for 256k derivatives

5.3 ISR macro

MCAL drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions:

1. OS is not used - AUTOSAR_OS_NOT_USED is defined:

- If USE_SW_VECTOR_MODE is defined:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, drivers' interrupt handlers are normal C functions and the prolog/epilog handle the context save and restore.

- If USE_SW_VECTOR_MODE is not defined:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, drivers' interrupt handlers must save and restore the execution context.

2. Freescale SemiconductorOS is used – AUTOSAR_OS_NOT_USED is not defined

```
#define ISR(IsrName) void OS_isr_##IsrName()
```

In this case, OS is handling the execution context when an interrupt occurs. Drivers' interrupt handlers are normal C functions.

3. Other vendor's OS is used – AUTOSAR_OS_NOT_USED is not defined. Please refer to the OS documentation for description of the ISR macro.

Please refer to the OS documentation for description of the ISR macro.

5.4 Other AUTOSAR modules - Dependencies

5.4.1 Development Error Tracer:

This module is necessary for enabling Development error detection. The API function used is Det_ReportError(). The activation / deactivation of Development error detection is configurable using 'GptDevErrorDetect' configuration parameter.

5.4.2 Diagnostic Event Manager:

This module is necessary for enabling reporting of production relevant error status. This is not used with current GPT implementation as the production relevant error codes are not present.

5.4.3 ECU Manager:

This module is used for processing the Wakeup notifications of GPT. Whenever the module is in 'Sleep' mode and a wakeup event occurs on a wakeup capable channel, it is reported to EcuM by calling EcuM_SetWakeupEvent () API through the EcuM_CheckWakeup()API. This is configurable using the 'GptReportWakeupSource' configuration parameter.

5.4.4 MCU module:

MCU module shall be initialized before using GPT .This module is required for setting the global prescaler value and to set the system clock frequency.

5.4.5 Configuration dependency to other module:

For generating configuration files of GPT, EcuM also is required as GPT refers to EcuM parameter. EcuM need to be configure first before generating configuration files of GPT.

Hence template files for EcuM are provided at

..\EcuM_TS_T2D13M10I1R0\autosar\EcuM.epd (Module Parameter Definition File – AUTOSAR Format)

..\EcuM_TS_T2D13M10I1R0\config\EcuM.xdm (Module Parameter Definition File – Tresos Format)

5.5 Exclusive areas to be defined in BSW scheduler

In the current implementation, GPT is using the services of Schedule Manager (SchM) for entering and exiting the critical regions. SchM implementation is done by the integrators of the MCAL using OS or non-OS services. For testing the ADC, stubs are used for SchM.

All GPT notification functions are called outside any critical region. Global variables updates are performed by ISRs before calling the user notification functions. So the GPT internal state is consistent at the moment of the notification call.

The following critical regions are used in the GPT driver:

5.5.1 EVEN_EXCLUSIVE_AREA

These areas are used in the functions Gpt_StartTimer/Gpt_StopTimer, channel is mapped to an even exclusive area according to formula $(2 * \text{channel})$ for Gpt_StartTimer/Gpt_StopTimer.

5.5.2 ODD_EXCLUSIVE_AREA

These areas are used in the functions Gpt_Enable/DisableNotification, channel is mapped to an even exclusive area according to formula $(2 * \text{channel} + 1)$ for Gpt_Enable/DisableNotification.

5.5.3 GPT_EXCLUSIVE_AREA_64

Used in function Gpt_SetMode, protects the register write and the global variable Gpt_Current_Mode write - this operation has to be atomic

Below is the table depicting the exclusivity between different critical region IDs from the GPT driver. If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

Exclusive areas to be defined in BSW scheduler

	GPT_EXCLUSIVE_AREA_00	GPT_EXCLUSIVE_AREA_01	GPT_EXCLUSIVE_AREA_02	GPT_EXCLUSIVE_AREA_64
GPT_EXCLUSIVE_AREA_00	X			
GPT_EXCLUSIVE_AREA_01		X		
GPT_EXCLUSIVE_AREA_02			X	
GPT_EXCLUSIVE_AREA_64				X

Chapter 6

Main API Requirements

6.1 Main functions calls within BSW scheduler

None

6.2 Calls to notification functions, callbacks, callouts

6.2.1 Call-back Notifications:

There are no call-back notifications defined inside the GPT driver.

6.2.2 User Notification:

The GPT Driver provides a notification per channel that is called whenever the defined time period is over.

The notifications can be configured as pointers to user defined functions. If notification is not desired,

‘NULL_PTR’ shall be configured.

An example of the syntax of this function is as follows:

```
void Gpt_Notification_<channel> (void)
```

An extern declaration of this function is available in Gpt_PBcfg.c. The function has to be implemented by the user.

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Config data:

```
#ifndef GPT_START_SEC_CONFIG_DATA_<ALIGNMENT>
#define GPT_START_SEC_CONFIG_DATA_<ALIGNMENT>
#define MEMMAP_ERROR
/*Memory Section for Post Build Data to be defined here. Example given in the
next line*/
#pragma ghs section rodata=".mcal_const_cfg"
#endif

#ifndef GPT_STOP_SEC_CONFIG_DATA_<ALIGNMENT>
#define GPT_STOP_SEC_CONFIG_DATA_<ALIGNMENT>
#define MEMMAP_ERROR
/*End of section to be mentioned here. Example given in the next line.*/
#pragma ghs section
#endif
```

For Code:

In case of the code is in Flash:

```
#ifndef GPT_START_SEC_CODE
#define GPT_START_SEC_CODE
#define MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#pragma ghs section text=".mcal_text"
#endif

#ifndef GPT_STOP_SEC_CODE
#define GPT_STOP_SEC_CODE
#define MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif
```

In case of the code is in RAM instead of Flash memory:

```
#ifndef GPT_START_SEC_RAMCODE
```

Sections to be defined in MemMap.h

```
#undef GPT_START_SEC_RAMCODE
#undef MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#pragma ghs section text=".ramcode"
#pragma ghs inlineprologue
#endif

#ifdef GPT_STOP_SEC_RAMCODE
#undef GPT_STOP_SEC_RAMCODE
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#pragma ghs nolineprologue
#endif
```

For Variables:

For variables that are never cleared and never initialized by start-up code:

```
#ifdef GPT_START_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef GPT_START_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#pragma ghs section bss=".mcal_bss"
#endif

#ifdef GPT_STOP_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef GPT_STOP_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif
```

For variables that have been initialized:

```
#ifdef GPT_START_SEC_VAR_INIT_<ALIGNMENT>
#undef GPT_START_SEC_VAR_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#pragma ghs section data=".mcal_data"
#endif

#ifdef GPT_STOP_SEC_VAR_INIT_<ALIGNMENT>
#undef GPT_STOP_SEC_VAR_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif
```

For Constant data:

```
#ifdef GPT_START_SEC_CONST_<ALIGNMENT>
#undef GPT_START_SEC_CONST_<ALIGNMENT>
#undef MEMMAP_ERROR
/*Memory Section for Constants to be defined here.*/
#pragma ghs section rodata=".mcal_const"
#endif
```



```

#ifdef GPT_STOP_SEC_CONST_<ALIGNMENT>
#undef GPT_STOP_SEC_CONST_<ALIGNMENT>
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif

```

Tables describe value range of shortcut **ALIGNMENT**:

Table 7-1. Range of <ALIGNMENT>

<ALIGNMENT>	Description
BOOLEAN	Used for variables and constants of size 1 bit
8	used for variables and constants which have to be aligned to 8 bit. For instance used for variables of size 8 bit or used for composite data types: arrays, structs and unions containing elements of maximum 8 bits.
16	used for variables and constants which have to be aligned to 16 bit. For instance used for variables of size 16 bit or used for composite data types: arrays, structs and unions containing elements of maximum 16 bits.
32	used for variables and constants which have to be aligned to 32 bit. For instance used for variables of size 32 bit or used for composite data types: arrays, structs and unions containing elements of maximum 32 bits.
UNSPECIFIED	used for variables, constants, structure, array and unions when SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. For instance used for variables of unknown size.

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

CONFIGURATION PARAMETER CONSIDERATIONS

Configuration parameter class for Autosar Gpt driver fall into the following variants as defined below:

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
GptDriverConfiguration			
	GptDevErrorDetect	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptReportWakeupSource	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptClockReferencePoint	Pre Compile parameter for all Variants of Configuration	Pre Compile
GptConfigurationOfOptApiServices			
	GptDeinitApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptEnableDisableNotificationApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptTimeElapsedApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptTimeRemainingApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptVersionInfoApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	GptWakeupFunctionalityApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
GptChannelConfigSet			
	GptSTMChannelPrescale	VariantPC or VariantPB	VariantPC or VariantPB
	GptSTMChannelPrescale_Alternate	VariantPC or VariantPB	VariantPC or VariantPB
GptChannelConfiguration			
	GptChannelId	VariantPC or VariantPB	VariantPC or VariantPB
	GptHwchannel	VariantPC or VariantPB	VariantPC or VariantPB
	GptChannelMode	VariantPC or VariantPB	VariantPC or VariantPB
	GptChannelTickFrequency	VariantPC or VariantPB	VariantPC or VariantPB

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	GptChannelTickValueMax	VariantPC or VariantPB	VariantPC or VariantPB
	GptChannelClkSrc	VariantPC or VariantPB	VariantPC or VariantPB
	GptChannelPrescale	VariantPC or VariantPB	VariantPC or VariantPB
	GptFreezeEnable	VariantPC or VariantPB	VariantPC or VariantPB
	GptEnableWakeUp	VariantPC or VariantPB	VariantPC or VariantPB
	GptNotification	VariantPC or VariantPB	VariantPC or VariantPB
	GptChannelClkSrcRef	VariantPC or VariantPB	VariantPC or VariantPB
	GptChannelPrescale_Alternate	VariantPC or VariantPB	VariantPC or VariantPB
GptWakeupConfiguration			
	GptWakeupSourceRef	VariantPC or VariantPB	VariantPC or VariantPB
GptNonAUTOSAR			
	GptEnableDualClockMode	VariantPC or VariantPB	Precompile
	GptChannelPrescale_Alternate	VariantPC or VariantPB	Precompile

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating GPT

- Generate the required GPT configurations. For more details refer to section [Files required for Compilation](#)
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section [Memory Allocation](#)
- Make sure all include files for compilation are as per section [ISR macro](#)
- Map the ISRs to their vector locations. For more details refer to section [Peripheral Hardware Requirements](#)
- Compile & build the GPT with all the dependent modules. For more details refer to section [Building the Driver](#) and [ISR to configure within OS – dependencies](#)



How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13GPTASR4.0R1.0.1
Revision 1.2